Abstract

Rationale: Imaging data driven artificial intelligence ~~(AI)~~ research in Nuclear Medicine ~~(NM)~~ increasingly depends on large-scale multi-modal 3D/4D datasets (PET/SPECT with CT/MRI) that must be resampled, aligned and quantitatively normalized without losing the spatial and clinical metadata required for physical interpretation (e.g. SUV, absorbed dose). Existing pipelines often struggle with (1) biobank-scale volume and I/O overhead, (2) execution speed and the "two-language" barrier, (3) metadata drift when images are converted to raw arrays (4) and end-to-end differentiability for novel physics-informed machine learning [?].

Methods: We developed `MedImages.jl`, a native Julia library that binds voxel data to spatial metadata (origin, spacing, direction) and clinical/temporal fields using a typed `MedImage` structure and a batched multimodal container (`BatchedMedImage`) for synchronized transforms. Volume-scale performance was evaluated on a 100-subject autoPET PET/CT alignment pipeline (target grid 512^3) comparing HDF5-based persistence to a Python caching pipeline. The execution speed was benchmarked on GPU for resampling, orientation changes, and fused affine transforms, and compared with SimpleITK (ITK-based C++ reference toolkit [?]). Differentiability was validated via gradient tests and learned spatial pipelines (inverse-rotation spatial transformer; organ-specific affine registration) and extended to physics-informed ~~neural network~~ dosimetry using a Universal Differential Equation (UDE) for ^{177}Lu-PSMA trained against Monte Carlo dose ground truths. Metadata integrity was assessed by SUV agreement with SimpleITK and by an end-to-end PET/CT transformation chain with lesion-based readouts.

Results: `MedImages.jl` achieved a 7.2× turnaround speedup versus caching in the 100-subject benchmark and up to 115× (resampling), 71× (orientation), and 135× (fused affine) GPU speedups versus CPU. SUVbw matched SimpleITK to 10^{-14} (double precision), and compounded transforms preserved Mean SUV (3.4913 → 3.4660; 0.73%) and lesion volume (52.55 → 51.86 cm^3; 1.32%), consistent with interpolation effects rather than metadata deterioration. Learned spatial transforms reduced reconstruction error by $> 65\%$, and UDE dosimetry refinement converged to MSE 4.14×10^{-4} versus Monte Carlo ground-truth dose maps [?].

Conclusion: `MedImages.jl` provides a unified, GPU-accelerated, metadata-aware, and differentiable foundation that addresses scalability, speed, differentiability, and metadata integrity for quantitative nuclear medicine preprocessing and AI/SciML workflows.

Key words: nuclear medicine; PET/CT; SUV; metadata; differentiable programming

1 Introduction

Nuclear medicine diagnostic data, such as PET and SPECT, provide quantitative measurements of biologic processes that support oncologic staging, assessment of therapy response, and planning of theranostic treatment. Correct processing and interpretation of functional imaging data require that measured uptake is assigned to the same patient-space coordinates as the anatomic refer-

ence (CT or MRI). Hybrid imaging therefore combines complementary modalities within a single, physically defined sampling space specified by image origin, grid or voxel spacing, and orientation.

The clinical landscape of targeted radioligand therapy has been fundamentally altered by the landmark VISION trial, which demonstrated significant improvements in overall survival (OS) and radiographic progression-free survival (rPFS) for patients receiving ^{177}Lu-PSMA-617 [?]. This success has led to its regulatory approval and the initiation of a new generation of clinical trials—including PSMAfore

Resolving the Nuclear Medicine "AI Chasm"

The AI chasm in nuclear medicine refers to the significant gap between the high expectations of artificial intelligence and its actual implementation in clinical practice [?]. This gap is largely driven by a scarcity of high-quality "AI-ready" databases, the difficulty of conducting large-scale multi-center evaluations, and the loss of critical quantitative data during preprocessing [?].

~~These limitations become even more pronounced in emerging Scientific Machine Learning (SciML) workflows, which increasingly rely on end-to-end optimization in which geometric preprocessing participates in training of AI models. In nuclear medicine , such workflows include learned registration and motion correction, data augmentation, and physics-informed models for image reconstruction or dosimetry. Although Python-based frameworks provide differentiable operations, practical composability across heterogeneous software components often requires committing the workflow to a single tensor ecosystem, creating friction when integrating third-party tools or classical numerical solvers [?, ?].~~

~~To address these limitations, we developed~~ MedImages.jl ~~, a Julia-based imaging ecosystem that converts high-level code into efficient machine instructions using just-in-time intermediate optimization and enables native implementations of geometric image operations. As a result, these operations can run efficiently on both CPUs and GPUs [?].~~

helps resolve this chasm through several key innovations:

- **Preventing "Metadata Drift" for Quantitative Fidelity**: A major barrier to clinical AI is that converting medical image objects into raw arrays for deep learning often detaches voxel intensities from their patient-space definitions and clinical metadata. MedImages.jl ~~binds~~ solves this by binding voxel data to spatial ~~and acquisition metadata through typed image representations, so that the typical for preprocessing geometric operations update the underlying physical description rather than operating on detached arrays. Core operators like resampling, affine warping, orientation handling, cropping and padding are implemented as native Julia kernels with transparent GPU acceleration , reducing practical bottlenecks in repeated 3D processing, supporting multimodal synchronization through batched representations.~~ metadata (origin, spacing, direction) and clinical fields (injection time, patient weight) using a typed `MedImage` structure. This ensures that critical physical interpretations—such as Standardized Uptake Value (SUV) and absorbed radiation dose—are preserved exactly at machine precision (matching reference standard tools to 10^{-14}).

- **Biobank-Scale Scalability**: Bridging the AI chasm requires massive,

multi-center datasets [?]. MedImages.jl utilizes a fused affine preprocessing strategy combined with HDF5-based storage and GPU acceleration to process data at extreme speeds. By collapsing spatial transformations (normalization, resampling, centering) into a single computational step, it achieves up to a 135× GPU speedup and reduces per-subject turnaround time to less than 200 milliseconds, solving the I/O and processing bottlenecks of biobank-scale workflows.

- **Synchronized Multimodal Alignment**: AI in nuclear medicine heavily depends on hybrid datasets (e.g., PET/CT, SPECT/CT). MedImages.jl uses a `BatchedMedImage` container that applies identical, synchronized geometric transformations across multiple modalities simultaneously without manual bookkeeping. This guarantees strict voxel-wise spatial correspondence across input channels, which is a fundamental prerequisite for training reliable multimodal neural networks.

~~The new framework is designed for interoperability with the broader Julia SciMLecosystem and with automatic differentiation libraries such as Zygote. jl~~

Enabling Biology- and Physics-Informed Neural Networks (SciML)

Scientific Machine Learning (SciML)—which intertwines known physical and biological laws with deep learning—is a major frontier for precision care, such as creating "theranostic digital twins" [?]. MedImages.jl accelerates this by making geometric and spatial operations natively differentiable.

- **End-to-End Differentiable Pipelines**: Because MedImages.jl is built natively in Julia, it integrates seamlessly with automatic differentiation frameworks like `Zygote.jl` and ~~Enzyme. jl, which allows for differentiable spatial pipelines [?]. This permits~~ `Enzyme.jl`. This means that geometric image operations ~~to~~ (resampling, rotation, cropping) can be incorporated directly into end-to-end ~~optimization, allowing spatial transformations to influence model training rather than remain external preprocessing steps. Julia further reduces software fragmentation by enabling image transformations, reconstruction algorithms, numerical solvers, and optimization routines to operate~~ model optimization without breaking the gradient flow or requiring non-differentiable C++ backends.

- **Triple-Branch Universal Differential Equations (UDEs) for Dosimetry**: MedImages.jl facilitates high-fidelity, physics-informed dosimetry by partitioning mechanistic physical laws from learned stochastic scattering. By unifying hardware acceleration, absolute metadata integrity, and differentiable physics within a single ~~coherent framework instead of across separate software layers that must be manually linked [?]. Since these compiler-level differentiation engines can act on arbitrary Julia code, gradients can propagate through solvers and GPU kernels without rewriting the underlying workflow in a specialized tensor framework [?].~~ language stack, MedImages.jl bridges the divide between the physics of image acquisition and the logic of AI inference.

In this work, we evaluate MedImages.jl across nuclear medicine–relevant benchmarks and experiments, both in isolation and in integration with complementary Julia libraries, with emphasis on (i) scalability for large PET/CT cohorts with high preprocessing demands, (ii) execution speed without the maintenance costs of a two-language architecture, (iii) metadata preservation to maintain the physical interpretability required for quantitative nuclear medicine, and (iv) differentiability of geometric operations for learned spatial transforms.

2 Methods

Figure 1: End-to-End Multi-Modal SciML Dosimetry Pipeline. Phase 1 shows raw clinical modalities in coronal view with typical acquisition misalignments (rotation and scaling discrepancies). Phase 2 demonstrates the result of unified preprocessing and batch alignment on a shared grid with soft-tissue windowed CT. Phase 3 illustrates the integration of these data into the Universal Differential Equation (UDE) framework. Phase 4 displays the final dosimetry inference and error residuals compared to Monte Carlo ground truth.

MedImages.jl was developed to address several practical requirements that are highly relevant in medical imaging, including preservation of image geometry and acquisition metadata, reliable handling of clinical image formats, efficient processing of large multimodal datasets, and support for optimization-based learning workflows. A central design feature is the `MedImage` data structure, which stores voxel values together with essential spatial information such as image position, voxel spacing, orientation, patient identity, and acquisition time. This avoids a common problem in AI-oriented workflows, where images are converted into generic tensors and thereby lose information needed to interpret them in physical space [**?**]. For robust reading and writing of clinical formats such as DICOM and NIfTI, MedImages.jl uses ITK through a wrapper interface, whereas spatial operations such as rotation, translation, and resampling are implemented directly in Julia language. This design combines the reliability of a mature medical imaging toolkit for file handling with the flexibility to use spatial operations in modern learning and optimization pipelines. For larger studies, the `BatchedMedImage` structure allows multiple images to be processed together while preserving the spatial properties of each individual volume. In combination with HDF5-based storage and GPU acceleration, this architecture reduces data-loading overhead, improves scalability, and supports workflows in which image transformations can be incorporated directly into model optimization.

Moreover, compatibility with the broader Julia SciML ecosystem allows MedImages.jl to interoperate with other scientific computing libraries and supports differentiable geometric operations for learned spatial transforms.

To demonstrate that MedImages.jl addresses the central computational and metadata-related challenges of large-scale medical image analysis, we conducted three proof-of-concept experimental series. ~~All experiments described are open-source and reproducible via the codebase provided in this repository~~An overview of the end-to-end multi-modal pipeline is illustrated in Figure 1, and the core framework functionalities are summarized in Table 1.

Table 1: Core functionalities and data structures of MedImages.jl for 3D medical image processing.

Functionality	**Description**
`MedImage` data structure	Unified medical image representation that stores voxel data together with clinical and vectorized spatial metadata.
`BatchedMedImage` structure	Batched representation for joint processing of multiple images while preserving the spatial properties of each individual volume.
Imaging-format-compatible I/O	Loading of standard medical image formats, including NIfTI and DICOM, via `ITKIOWrapper`.
HDF5 support	Fast loading and saving in big-data compatible format, facilitating efficient intermediate storage in repeated processing and deep-learning workflows.
GPU acceleration	Transparent GPU support for spatial transformations through `KernelAbstractions.jl`, enabling accelerated parallel execution of preprocessing and geometric operations.
Resampling	Native Julia functions for resampling, enabling voxel-wise correspondence across modalities.
Spatial transformations	Native Julia functions for manipulation of spatial properties of 3D image volumes, including `rotate_mi`, `translate_mi`, `scale_mi`, `resample_to_spacing`, `crop_mi`, and `pad_mi`.
Orientation management	Tools for handling, preserving, and converting image orientations across neuroimaging and medical imaging coordinate systems.
Automatic differentiation	Differentiable implementation of spatial transformations with well-defined gradients through `Zygote.jl` and `Enzyme.jl`.

Experimental Series 1: Evaluation of Scalability and Execution Speed

For large-scale imaging studies, we evaluated and compared MedImages.jl ~~with~~ against contemporary frameworks with respect to two key requirements: scalability to ~~large~~ massive multimodal cohorts and raw runtime of core spatial preprocessing operations in repeated deep-learning workflows.

To assess scalability and computational overhead, we designed a 100-case preprocessing benchmark based on the autoPET dataset [**?**]. This setup ~~was intended to reflect a realistic large-cohort scenario in which multiple~~ reflects a realistic biobank-scale scenario where serial PET/CT studies must be repeatedly loaded ~~, harmonized and~~ and geometrically prepared during iterative model training.

The main preprocessing task was multimodal spatial alignment of heterogeneous PET and CT volumes on a common isotropic reference grid of ~~512 × 512 × 512~~ 512^3 voxels. This harmonization is essential for multimodal neu-

ral networks, which require strict voxelwise spatial correspondence across input channels. ~~In practical terms, alignment means transforming both modalities into the same physical coordinate system such that a voxel in position (i,j,k) of the PET volume corresponds to the same anatomical location as the voxel in position (i,j,k) of the CT volume.~~

This benchmark was divided into two computational phases per subject: first, loading the preprocessed case from cache, and second, applying the remaining ~~computational~~ transformations required to prepare the tensor data for training.

Within the MedImages.jl pipeline, HDF5-based storage was combined with a fused affine preprocessing strategy to reduce computational overhead. Rather than performing orientation normalization, voxel spacing adjustment, and spatial centering as separate steps, these operations were merged into a single affine transformation ~~. This allowed each volume to be resampled in a single step, thus reducing repeated interpolation and minimizing data movement.~~ Pass.

MedImages.jl was compared with MONAI [?] using its PersistentDataset mechanism[?], thus assessing not only the ~~computational performance at the~~ framework-level performance, but also the effect of different data persistence strategies. Performance was evaluated under repeated training conditions, beginning from epoch 2 onward~~, under the assumption that the data had already undergone initial preprocessing steps and had been stored in cache~~.

To assess raw computational speed independently of the caching benchmark, ~~we additionally measured~~ isolated spatial operations were measured across multiple scales. These tests included resampling, orientation changes, and affine transformations~~, which represent some of the most common geometric operations in multimodal medical image preprocessing~~. Benchmarks were performed on single volume of size ~~256 × 256 × 128~~ $256 \times 256 \times 128$ using both CPU and GPU backends. SimpleITK [?] was evaluated on CPU, reflecting its typical use~~for such geometric operations~~, whereas MedImages.jl was evaluated on both CPU and GPU ~~in order~~ to quantify the ~~effect of hardware acceleration on the same class of tasks~~hardware acceleration effect.

Both the 100-subject caching pipeline benchmark and the isolated raw computational speed benchmarks ~~(resampling, orientation changes, fused affine transformations)~~ were executed on ~~same hardwareconfigurations~~strictly identical hardware: an Intel Core Ultra 9 285K processor ~~for CPU-bound computations~~ and an NVIDIA RTX 3090 GPU~~for hardware-accelerated processing. ”~~ .

.

Experimental Series 2: Verification of Metadata Integrity and Quantitative Clinical Fidelity

To evaluate the quantitative accuracy of MedImages.jl, Standardized Uptake Value normalized by body weight (SUVbw) was compared against a reference implementation in SimpleITK [?]. SUVbw was computed using same PET image data from autopet dataset and corresponding clinical and temporal metadata in both frameworks. The comparison was performed in double-precision floating point arithmetic~~, and numerical agreement between both implementations was assessed~~ . For the fused affine interpolation kernel, the numerical results obtained on the CPU ~~by reference Python library~~ and GPU were compared directly, and numerical agreement was strictly verified within a tolerance of 10^{-4} to guarantee floating-point consistency across backends.

An end-to-end consistency experiment was subsequently conducted on ~~the~~

a subset of autoPET dataset to assess the preservation of quantitative fidelity under a sequence of spatial transformations. The processing pipeline consisted of (1) segmentation of lesion masks on the original image data, (2) conversion of PET intensities to SUV, (3) resampling of PET volumes and corresponding lesion masks to the native CT coordinate space, (4) resampling to an isotropic voxel spacing of 2.0 mm, (5) application of a ~~45°~~ 45° rotation around the z-axis, and (6) translation by 10 voxels along the x-axis. Nearest-neighbor interpolation was applied to segmentation masks, while linear interpolation was used for continuous PET and CT intensity data. Quantitative evaluation was performed by measuring mean SUV and total lesion volume within the segmentation mask before and after the transformation sequence (see Figure 2).

Figure 2: End-to-end consistency experiment illustrating the sequence of spatial transformations applied to the autoPET dataset. The pipeline demonstrates the preservation of quantitative fidelity during sequential spatial manipulation steps including conversion to SUV, resampling to native CT space, isotropic resampling, rotation, and translation.

To evaluate the capability of MedImages.jl for handling complex theranostic imaging data, a multi-modal processing experiment was conducted using a dataset consisting of four imaging modalities from a single patient who underwent a radioligand therapy. The data set included: (1) [^{177}Lu]Lu-PSMA SPECT with attenuation correction (AC) (2) [^{177}Lu]Lu-PSMA SPECT without attenuation correction (NAC), (3) a voxel-wise absorbed dose map, and (4) a CT volume serving as anatomical reference. Each modality was assumed to originate from independent acquisition and reconstruction pipelines and therefore retained its own voxel grid, data type, and spatial metadata, including voxel spacing, image origin, and orientation. All four modalities were loaded into a single BatchedMedImage data structure. During this step, the original voxel data and modality-specific spatial metadata were preserved without prior resampling or harmonization. Subsequently, a geometric transformation was applied to the entire batch. A spatial rotation of 45° around the Z-axis (axial plane) was executed simultaneously across all four modalities using a single batched transformation call, followed by a linear translation of 10 voxels along the X-axis to verify spatial origin tracking.

The outcome of the transformation was evaluated qualitatively by visual inspection of corresponding axial slices before and after transformation. Particular attention was given to the preservation of spatial alignment between functional tracer distribution and anatomical structures. The consistency of spatial relationships across modalities after transformation served as the primary validation criterion. The exact methodology parameters for this experiment are detailed in Table 4 in the Appendix.

~~**Experimental Series 3: Verification of Differentiability and Learned Spatial Transformations**~~ All experiments described are open-source and reproducible via the codebase provided in this repository.

Experimental Series 3: Verification of Differentiability and Learned Spatial Transformations

To determine whether spatial image operations remained mathematically trainable and could therefore be used reliably in end-to-end learning pipelines, gradient propagation was numerically evaluated across representative geometric transformations implemented in MedImages.jl. Reverse-mode gradients were computed with Zygote.jl for translation, padding, cropping, and fused affine interpolation. For translation and padding, correctness was assessed by verifying that the gradient remained constant throughout the transformed image domain. For cropping, the gradient pattern was examined to confirm preservation within the retained image region and suppression outside the cropped area. For the fused affine interpolation kernel, the gradients obtained on the CPU and GPU were compared directly, and numerical agreement was verified within a tolerance of ~~10 ^ -4.~~ 10^{-4}.

To further assess end-to-end differentiability in a trainable setting, a learned inverse rotation experiment was conducted using synthetic single-case batched image configurations. A synthetic 3D structural line image was generated on a strictly confined $16 \times 16 \times 16$ voxel grid to ensure strong spatial gradient signals. A total of 120 batched sample configurations (100 for training, 20 for testing) were generated by applying random three-axis sequential rotations within an angular range of $\pm 30°$. A lightweight 3D Convolutional Neural Network (CNN) combined with a Multi-Layer Perceptron (MLP) head was then trained using gradient descent for 20 epochs to predict the inverse Euler rotation parameters

directly from the spatial grids.

2.1 ~~High-Fidelity Dosimetry via Triple-Branch Universal Differential Equations~~

3 Results

~~In order to showcase the example of the end to end practical implementation utilizing Medimages. jl Authors had designed and executed the Dosimetry experiment utilizing unique SciML julia library .~~ All results in scalability, computational performance, metadata integrity, quantitative fidelity, and differentiability are summarized in Table 2.

3.0.1 ~~Model Architecture: The Triple-Branch UDE~~

Evaluation of Scalability and Processing Speed

In iterative training scenarios, MedImages.jl demonstrated substantially reduced processing times compared to established frameworks, decreasing total turnaround per subject from several hundred milliseconds (MONAI [**?**]) to well below one-tenth of a second. The greatest improvements were observed during the transformation stage, where the fused affine pipeline reduces repeated interpolation and data movement by performing multiple spatial operations in a single computational step.

GPU acceleration further amplified performance gains across core spatial operations. For medium-sized volumes ($256 \times 256 \times 128$), resampling, orientation changes, and affine transformations achieved speedups exceeding one to two orders of magnitude compared with CPU execution, with additional improvements over standard CPU-based libraries. Notably, while frameworks like MONAI also utilize GPU acceleration for transformations, their typical implementation relies on a sequence of individual operations (e.g., orientation normalization followed by resampling). In our benchmarks, this sequential approach resulted in a computation time of approximately 500 ms per subject on the GPU. In contrast, MedImages.jl's fused affine kernel mathematically collapses the entire transformation stack into a single Pass, achieving a 10× computational speedup on the same hardware. In large-scale settings (512^3 target grid, $n = 100$), per-subject processing times remained below 200 ms, confirming suitability for high-throughput and biobank-scale workflows.

Evaluation of Metadata Integrity and Quantitative Fidelity

Quantitative validation demonstrated numerical agreement with the reference standard at machine precision for SUV calculations, with differences on the order of 10^{-14}. Across compound spatial transformations, mean SUV changed only minimally (from 3.49 to 3.47), and lesion volume remained stable (from approximately 52.6 cm^3 to 51.9 cm^3), corresponding to relative deviations below 1–2%. These differences are consistent with expected interpolation effects and do not indicate degradation of clinical metadata.

Evaluation of Differentiability and Learned Transformations

Gradient propagation across spatial operations followed expected analytical behavior, with identity-preserving transformations maintaining uniform gradients and spatially selective operations producing region-dependent responses. Numerical agreement between CPU and GPU gradient computations was maintained within a tolerance of 10^{-4}.

In learned transformation experiments, optimization of geometric parameters resulted in a reduction in reconstruction error exceeding 65% within 20 training iterations, demonstrating effective end-to-end training. Affine registration tasks showed stable convergence using GPU-based gradient optimization, confirming that complex spatial transformations can be integrated directly into differentiable learning pipelines without external implementations. Figure 3 visualizes the learned inverse rotation trajectory.

Figure 3: Learned inverse rotation trajectory across training epochs, demonstrating end-to-end spatial differentiability. The model progressively aligns the uncorrected volume (red) towards the gold standard (blue).

Table 2: Summary of results across all experimental series.

Experiment Series	Metric / Task	Result	F
Series 1: Scalability and Execution Speed			
Data loading	Load from cache	**40 ms**	
Computation	Transform (compute)	**50 ms**	
Overall	Total turnaround	**90 ms**	
GPU performance	Resampling	**115× faster (GPU vs CPU)**	
GPU performance	Orientation changes	**71× faster (GPU vs CPU)**	
GPU performance	Fused affine transform	**135× faster (GPU vs CPU)**	
Large-scale pipeline	512^3 volumes (100 cases)	**¡ 200 ms per subject**	
Series 2: Metadata Integrity and Quantitative Fidelity			
SUV validation	SUVbw agreement	$\approx 10^{-14}$	Matche
Transformation consistency	Mean SUV deviation	**0.73%**	
Transformation consistency	Lesion volume deviation	**1.32%**	
Series 3: Differentiability and Learned Transformations			
Gradient validation	Identity transforms	**Gradient = 1**	Co
Gradient validation	Cropping	**1 (inside), 0 (outside)**	Expected
Gradient validation	CPU vs GPU consistency	$< 10^{-4}$ **difference**	Numerical co
Learning experiment	Inverse rotation (CNN)	**¿ 65% MSE reduction**	
Registration	Affine optimization	**Converged**	GPU
Improved UDE dosimetry	Final voxel-wise dose error	**MSE** $= 4.14 \times 10^{-4}$	Monte

4 High-Fidelity Dosimetry via Triple-Branch Universal Differential Equations

4.1 Model Architecture: The Triple-Branch UDE

The proposed model evolves the standard dosimetry framework into a ~~simplified neural~~ Universal Differential Equation (UDE) that partitions mechanistic physical laws from learned stochastic scattering. The core innovation is the expansion to a **Triple-Branch Input** architecture:

1. **Activity Branch** (A_{total}): Tracks the biological time-varying radiopharmaceutical concentration derived from the compartmental ODE states (~~$A_{free} + A_{bound}$~~free and bound activity).

2. **Density Branch** (ρ): Incorporates voxel-specific mass-density derived from CT ~~Hounsfield Units~~data.

3. **Gradient Branch** ($|\nabla\rho|$): ~~A high-pass geometric prior that explicitly signals~~ Acts as a geometric prior to explicitly signal tissue-bone and tissue-air boundaries, ~~allowing the~~ which allows the neural network to learn asymmetric Compton scattering.

4.1.1 ~~Numerical Stability: Balanced Unit Training~~

4.2 Numerical Stability: Balanced Unit Training

To ensure convergence on GPU architectures, we implemented **Balanced Unit Training**. SPECT activity variables are internally re-scaled by 10^{-6} to bring the IVP (Initial Value Problem) into an $O(1)$ range. The physical dose constant (C_{conv}) is balanced accordingly, ensuring the integrated output maintains its physical identity in Grays (Gy).

4.3 Physical Formulation

The system is defined by the following coupled system of equations:

$$\frac{d}{dt}\text{Dose}(t) = \text{softplus}\left(\frac{A_{total}(t) \cdot C_{conv}}{\rho \cdot V} + \mathcal{N}_\theta(A_{std}, \rho_{std}, |\nabla\rho|_{std})\right) \quad (1)$$

Where $\mathcal{N}_\theta$ is a 3D Convolutional Neural Network learning the corrective residual to the analytical dose rate.

The formulation integrates three distinct computational logic blocks:

1. ~~Analytical~~ Integrating Classical Physics ~~Term~~with Deep Learning: The ~~component $\frac{A_{total}(t) \cdot C_{conv}}{\rho \cdot V}$ represents the local energy deposition . Here, $A_{total}(t)$ is the sum of free and bound compartmental activity, C_{conv} is the dose conversion constant, and the denominator $\rho \cdot V$ calculates the voxel mass (Density × Volume) , ensuring the result maintains the physical unit of Grays (Gy)~~ software formulates energy deposition analytically (calculating dose rate via $\frac{A_{total}(t) \cdot C_{conv}}{\rho \cdot V}$) while simultaneously using a 3D Convolutional Neural Network ($\mathcal{N}_\theta$) as a learned corrective residual for complex stochastic effects.

2. **Neural Network Residual**: The term $\mathcal{N}_\theta(A_{std}, \rho_{std}, |\nabla\rho|_{std})$ acts as a learned correction for stochastic effects such as asymmetric Compton scattering. By utilizing a Triple-Branch input (standardized activity, density, and the density gradient $|\nabla\rho|$), the model explicitly identifies tissue boundaries where classical analytical kernels typically fail.

3. **Differentiable Positivity**: The entire rate is wrapped in a *softplus* activation function. This ensures that the ~~generated dose remains~~ predictions remain physically non-negative while providing a smooth, continuously differentiable surface required for stable backpropagation through the ODE solver during end-to-end SciML training.

5 ~~Results~~

~~All results in scalability, computational performance,~~ By unifying hardware acceleration, absolute metadata integrity, ~~quantitative fidelity, and differentiability~~ and differentiable physics within a single language stack, MedImages.jl bridges the divide between the physics of image acquisition and the logic of AI inference.

4.1 Results and Comparative Analysis

The Improved UDE was evaluated on a full-body validation cohort using a sliding-window reconstruction (64^3 patches, stride 32). The performance of the Triple-Branch UDE was benchmarked against traditional static physics models, state-of-the-art pure deep learning architectures (like DblurDoseNet [?]), and the gold-standard Monte Carlo (MC) radiation transport simulations [?]. Quantitative performance metrics are summarized in Table ~~2~~3, and a qualitative comparison of the corresponding dose maps is presented in Figure 4.

~~Evaluation of Scalability and Processing Speed~~

~~In iterative training scenarios, MedImages. jl demonstrated substantially reduced processing times compared to established frameworks, decreasing total turnaround per subject from several hundred milliseconds (MONAI) to well below one tenth of a second. The greatest improvements were observed during the transformation stage, where the fused affine pipeline reduces repeated interpolation and data movement by performing multiple spatial operations in a single computational step.~~

Table 3: Full-Body Performance Comparison (Pearson Correlation and Mean Absolute Error)

Model	**Mean Pearson (r)**	**MAE (Gy)**
Analytical Baseline (Static)	0.8249	700.43
DblurDoseNet (Pure DL)	0.5566	1102.34
Spect0Net	0.6124	955.12
SemiDose	0.6401	912.88
Original UDE (2-Branch)	0.9031	405.77
UDE IMPROVED (Triple-Branch)	**0.9221**	**359.49**

~~GPU acceleration further amplified performance gains across core spatial operations. For medium-sized volumes ($256 \times 256 \times 128$)~~

Figure 4: Multi-model comparative dosimetry. Qualitative evaluation of the proposed Triple-Branch UDE framework against purely analytical baseline and deep-learning (CNN) models. Error residuals are computed relative to the gold-standard Monte Carlo simulated absorbed dose maps, demonstrating superior performance and reduced hallucination artifacts.

4.2 Comparative Efficacy and Clinical Viability

To objectively evaluate the UDE model, it must be contrasted with the prohibitive computational latency of the gold-standard Monte Carlo simulations. While MC particle tracking represents the absolute pinnacle of physical accuracy, generating a high-fidelity, ~~resampling, orientation changes, and affine transformations achieved speedups exceeding one to two~~ low-noise dose map for a single single-bed SPECT study requires approximately **48 hours (2 days)** of continuous processing on standard clinical CPU infrastructure [**?**]. This creates an insurmountable bottleneck for real-time adaptive therapy [**?**].

In contrast, while pure convolutional networks like DblurDoseNet can perform inference in approximately 3 seconds [**?**], they operate as "black box" models

prone to catastrophic hallucination in out-of-distribution (OOD) scenarios, such as when crystal thickness or reconstruction algorithms shift across centers. The UDE framework establishes a middle ground: achieving near real-time inference (measured in seconds to tens of seconds) while rigorously anchoring predictions in exact physical laws.

The UDE solves the inherent "black box" crisis of modern artificial intelligence in healthcare. By explicitly programming the known mechanisms of radiation decay and fluid compartment transport into the system, the model structurally prevents physiological violations. It relegates the neural network to its optimal role: approximating the complex, non-linear patient heterogeneities—such as unique tumor micro-environment clearance rates—that simply cannot be described by classical analytic functions [**?**].

Because the UDE is irrevocably anchored by invariant physical laws, its parameter space is vastly reduced and highly constrained compared to unconstrained models like DblurDoseNet. This physics-guided constraint makes UDEs exceptionally robust to dataset shifts, a monumental advantage in heterogeneous multi-center clinical trials [**?**]. Furthermore, the UDE's ability to extrapolate accurate multi-day time-activity curves (TACs) from limited boundary conditions makes it the only theoretically sound choice for Single-Time-Point (STP) dosimetry, which is rapidly becoming the desired clinical standard to minimize patient burden.

Finally, by partitioning mechanistic laws from learned biological clearance rates (μ_{bio}), the UDE model provides a level of interpretability and regulatory trust absent in pure DL architectures [**?**]. Final outputs can be decomposed into their constituent physical and learned components, allowing medical physicists to interrogate the biological parameters for physiological plausibility (e.g., ensuring non-negative clearance rates) before clinical decision-making. The proposed paradigm thus stands as the apex model for personalized radiopharmaceutical dosimetry in the modern theranostic era.

4.3 Computational Performance: SciML vs. Pure Deep Learning

While the Triple-Branch UDE achieves superior accuracy through physics-informed constraints, integrating a system of ordinary differential equations (ODEs) inherently increases computational complexity compared to purely feed-forward deep learning models.

To properly benchmark the UDE approach and address the multi-language landscape, we implemented mathematically identical Triple-Branch UDEs (same 3D CNN architectures and ODE formulations) across three frameworks: Native Julia (SciML + Lux.jl + Zygote), PyTorch (`torchdiffeq`), and JAX (`diffrax` via Equinox). We additionally included a hybrid Python approach (`scipy.integrate.solve_ivp` interacting with a PyTorch CNN). To ensure isolation of purely computational capabilities and to allow for fully open-source reproducibility without relying on protected patient data, all framework-level execution and memory benchmarks were performed using synthetic random tensors simulating a 32^3 voxel patch. All tests were run on an NVIDIA H100 PCIe GPU, explicitly excluding Just-In-Time (JIT) compilation overhead to ensure absolute equivalency.

Inference Speed (300-hour integration):

- **JAX (`diffrax`): 9.4 ms** (Highest performance due to end-to-end XLA

compilation of the ODE solver and neural network).

- **PyTorch (`torchdiffeq`): 24.0 ms.**
- **Julia SciML (`Euler`): 48.4 ms** (Highly competitive native performance without requiring strict XLA constraints).
- **Hybrid Python (`scipy.integrate`): 675.2 ms** (Severely bottlenecked by CPU-GPU data transfer overheads).

Training Performance (Forward + Backward Pass) & Memory Analysis: Inspired by the architectural evaluations in `SciMLBenchmarks.jl`, we conducted a deeper analysis into the memory scaling and adjoint sensitivity methods utilized by the different frameworks during backpropagation through the ODE solver:

- **JAX**: **52.7 ms** per step. Peak memory: **197.4 MB**. (JAX achieves extraordinary efficiency by compiling the entire reverse-mode ODE solve into a single optimized XLA graph).
- **PyTorch**: **62.9 ms** per step. Peak memory: **1.37 GB**. (High memory consumption is observed due to the standard practice of retaining the large 3D CNN computational graph across all ODE solver steps during backpropagation).
- **Julia SciML (BacksolveAdjoint)**: **478.8 ms** per step. Peak memory: **≈ 2.45 GB**. (By applying insights from `SciMLBenchmarks.jl` and using a `BacksolveAdjoint`, the solver re-integrates the ODE backward in time instead of storing dense checkpoints. Slower training times currently reflect the overhead of `Zygote.jl` tracking complex neural network VJPs within the ODE solver loop, compared to the mature C++ backends of PyTorch/JAX).
- **Julia SciML (EnzymeVJP Trial)**: Following best practices from `SciMLBenchmarks.jl` for neural ODEs, we attempted to compile the reverse-mode ODE adjoint directly using `Enzyme.jl`. While Enzyme provides unmatched performance for scalar and small-scale array codes, attempting to JIT-compile the reverse-mode AD graph for a 3D neural ODE on the GPU resulted in timeouts and hard CUDA memory crashes after over an hour of compilation. This empirically highlights that while theoretically superior, compiling highly-branched, large-scale Neural PDEs with Enzyme on GPUs currently exceeds practical hardware compiler limits, remaining an active area of development.

Although the native Julia UDE approach is slower than JAX/XLA in this specific fixed-step, GPU-bound training setup, its inference time (48.4 ms per patch) remains highly competitive and easily scalable to full-body reconstruction (under 3 minutes per patient). It is important to contextualize this performance within the broader landscape of differential equation solvers.

As extensively documented by the `SciMLBenchmarks.jl` suite (specifically the *Multi-Language Wrapper Benchmarks* comparing Julia, SciPy, MATLAB, and deSolve), native Julia ODE solvers systematically outperform multi-language wrapper packages by orders of magnitude ~~compared with CPU execution, with~~

~~additional improvements over standard CPU-based libraries. In large-scale settings (512^3 target grid, $n = 100$), per-subject processing times remained below 200 ms, confirming suitability for high-throughput and biobank-scale workflows.~~

~~Evaluation of Metadata Integrity and Quantitative Fidelity~~

~~Quantitative validation demonstrated very high numerical agreement with the reference standard at machine precision for SUV calculations, with differences on the order of 10^{-14}. Across compound spatial transformations, mean SUV changed to a negligible extent (from 3.49 to 3.47),~~ on standard stiff and non-stiff problems. In general cases, SciML's dramatic superiority stems from the elimination of cross-language interpreter overhead (e.g., Python repeatedly calling C++ solver backends), the ability to aggressively JIT-compile the entire solver loop, and ~~lesion volume remained stable (from approximately 52.6 cm^3 to 51.9 cm^3), corresponding to relative deviations below 1–2%. These differences are consistent with expected interpolation effects and do not indicate degradation of clinical metadata.~~

~~Evaluation of Differentiability and Learned Transformations~~

~~Gradient propagation across spatial operations followed expected analytical behavior, with identity-preserving transformations maintaining uniform gradients and spatially selective operations producing region-dependent responses.Numerical agreement between CPU and GPU gradient computations was maintained within a tolerance of 10^{-4}~~the use of highly optimized stack-allocated arrays for small-to-medium physical systems.

~~In learned transformation experiments, optimization of geometric parameters resulted in a reduction in reconstruction error exceeding 65% within 20 training iterations, demonstrating effective~~ Our specific neural dosimetry case diverges from these general benchmarks because the computational load is overwhelmingly dominated by massive, static 3D convolutions on the GPU rather than the ODE solver's internal stepping logic. This architectural pattern—a fixed-step solver wrapping a static, massive tensor graph—is uniquely suited to JAX's end-to-end ~~training. Affine registration tasks showed stable convergence using GPU-based gradient optimization, confirming that complex spatial transformations can be integrated directly into differentiable learning pipelines without external implementations~~XLA compiler, which fuses the entire operation into a monolithic GPU kernel, yielding the observed 9.4 ms inference speed.

However, based on the fundamental limits exposed by the SciML multi-language benchmarks, Julia SciML's performance superiority is projected to re-emerge rapidly as the dosimetry model approaches physiological reality. When the biological system becomes heavily stiff (e.g., modeling rapid receptor saturation, non-linear pharmacokinetics, or instantaneous drug injection pulses requiring discrete callbacks), simple fixed-step solvers like Euler fail numerically, and advanced adaptive-step solvers are required. Crucially, the Julia implementation remains the only one capable of natively handling the stiff versions of these equations (e.g., seamlessly switching to advanced solvers like `KenCarp4` or `Rodas5`) without requiring bespoke C++ wrappers.

Implementing and differentiating through complex, dynamically branching stiff solvers in JAX or PyTorch is fundamentally prohibitive without writing custom CUDA kernels. Thus, the MedImages.jl and SciML framework trades raw feed-forward speed on static GPU graphs for unparalleled physical accuracy and solver flexibility, which is critical for maintaining the physical validity of complex biological dosimetry models where concentration gradients shift abruptly.

4.4 Limitations and Simplifications

The primary objective of this experiment was to serve as a proof of concept, demonstrating the validity and feasibility of the UDE-based approach in Julia. While the UDE model definitively achieves the highest accuracy among the tested baselines, several physical abstractions were employed to simplify the initial implementation. To maintain scientific rigor and satisfy future clinical peer review, we acknowledge the following physical realities that the current framework abstracts: 1. **Static Anatomy**: The model assumes a rigid day-0 CT geometry, neglecting 4D respiratory motion. 2. **Isotropic PK**: Diffusion between neighboring voxels is currently ignored in the interstitial compartment. 3. **Receptor Dynamics**: Receptor capacity (B_{MAX}) is treated as a constant, neglecting radiation-induced receptor degradation ("stunning"). 4. **Transport Lumping**: Local β^- and penetrating γ transport are lumped into a single neural correction term rather than a dual-kernel separation.

5 ~~Disscusion~~

Despite these necessary simplifications, the experiment successfully proves the validity of the approach: intertwining known physics with deep learning via Julia's SciML stack yields substantially superior dosimetry results compared to pure deep learning models and classical analytical convolutions.

5 Discussion

As biomedical imaging increasingly embraces complex multimodal, multidimensional, and longitudinal studies, alongside scientific machine learning (SciML), the interoperability and performance of data management frameworks become critical. Our results indicate that MedImages.jl's metadata handling paradigm, combined with seamless integration into the Julia scientific ecosystem, provides a highly performant and differentiable framework that systematically addresses the ~~three core challenges outlined in the Introduction~~challenges of large-scale clinical AI.

Addressing ~~Challenges 1 the~~ Speed and ~~2 (~~Volume ~~and Speed): The Advantage of Native Fused Kernels~~Bottlenecks

Accurate metadata alignment is a fundamental requirement for contemporary multimodal image analysis. For example, training convolutional neural networks (CNNs) on combined PET/CT data requires multi-channel tensors in which voxel (i,j,k) in the PET dataset corresponds precisely to the same anatomical

location as corresponding voxel in the CT dataset. Owing to the large size of 3D voxel-based imaging data, the necessary continuous spatial resampling may ~~, as observed in our experiments,~~ represent the principal computational bottleneck in deep learning workflows.

Integration of metadata within the `MedImage` structure directly translates into performance gains observed in our benchmarks. The 7.2× turnaround advantage observed in high-frequency training loops is primarily attributed to MedImages.jl's integration with HDF5~~—~~which bypasses the serialization costs inherent in many Python caching mechanisms~~—~~and its single-pass Fused Affine kernel. Notably, while established frameworks like MONAI [**?**] also offer GPU-accelerated transformations, they typically apply operations as a sequence of separate kernels (e.g., orientation followed by resampling). By mathematically collapsing the entire spatial transformation stack ~~(normalization, resampling, centering, etc.)~~ into a single ~~CUDA execution~~Pass, MedImages.jl eliminates the memory traffic and synchronization overhead associated with sequential operation queues, providing a highly efficient solution for biobank-scale deployments.

~~In contrast, MedImages.jl leverages Julia's type system to bind metadata directly within the `MedImage` struct. As summarized in Table **??**, by removing the runtime overhead associated with dictionary-based metadata access, this approach contributes to the improved performance observed in our benchmarks. Furthermore, the native implementation of core geometric transformations in Julia enables MedImages.jl to maintain the automatic differentiation graph across processing steps. This is particularly relevant for imaging workflows that incorporate scientific machine learning, where gradient information may be lost when operations are passed to non-differentiable compiled backends cite.~~

~~The Primacy of Clinical and Temporal Metadata~~

~~The lack of standardization in medical imaging preprocessing pipelines can pose a reproducibility challenge [**?**]. In such settings, metadata drift, defined as the decoupling of spatial or clinical metadata from voxel data, may introduce clinically relevant errors. For example, it can lead to displacement of treatment margins in radiotherapy or compromise quantitative interpretation in PET/CT when decay-related timing information is lost \cite. Quantitative nuclear medicine is particularly dependent on the integrity of clinical and temporal metadata, as standardized uptake value (SUV) calculations explicitly require accurate injection and acquisition times. In theranostic workflows, this requirement must be maintained while applying identical geometric transformations across multiple co-registered modalities, such as ^{177}LuLu-PSMA SPECT-AC/NAC, absorbed-dose maps, and CT.~~

~~MedImages.jl addresses this by design: the `BatchedMedImage` allows single-call, batched geometric transforms to be applied consistently across PET and CT modalities without manual bookkeeping. Our cross-platform validation confirms that the SUV conversion factors extracted by MedImages.jl achieve mathematical identity with the established SimpleITK benchmarks to a precision of 10^{-14}. This numerical equivalence ensures that researchers transitioning to Julia can maintain absolute continuity with legacy clinical findings while gaining the performance required for massive biobank-scale analysis.~~

As demonstrated in the last experiment in these series, the batched data format shows the ability to apply consistent geometric transformations across multiple functional (SPECT, Dosemap) and anatomical (CT) data in a single operation. It ensures precise pixel-wise spatial alignment, which is essential for image augmentation and multimodal registration tasks, e.g. for maintaining consistent dosimetric outcomes.

Addressing Challenge 3 (Differentiability): Navigating the Framework Ecosystem

The Julia Differentiable Imaging Physics The Julia ecosystem is uniquely positioned for SciML because its automatic differentiation frameworks (Zygote.jl, Enzyme.jl`Zygote.jl`, `Enzyme.jl`) can differentiate through arbitrary native code, including custom spatial transformations. Nuclear medicine imaging is inherently quantitative, but its key readouts (e.g., SUV, time–activity curves, absorbed dose) are highly sensitive to geometric preprocessing such as resampling , rotation, and registration. In practice, these operations must often be integrated into AI pipelines for motion correction, longitudinal alignment, and learned preprocessing, where transform parameters are optimized jointly with network weights. However, many commonly used imaging backends execute spatial resampling outside the automatic differentiation graph, preventing end-to-end training through geometric steps[?]. To demonstrate that . MedImages.jl enables fully differentiable volumetric spatial operationssuitable for nuclear medicine workflows, we implement several SciML validation tests, allowing geometric image operations (resampling, rotation, cropping) to be incorporated directly into model optimization without breaking the gradient flow.

When established Python frameworks remain the pragmatic choice. For teams with substantial legacy Python codebases, switching frameworks carries non-trivial migration costs. MONAI provides a mature library of ready-made components—pretrained segmentation networks, standardized data-loading pipelines, and extensive augmentation transforms—that can be deployed with minimal custom code. Similarly, SimpleITK offers a comprehensive catalogue of classical filters whose correctness has been validated across decades. When the research question can be answered with an existing pipeline and the primary requirement is rapid prototyping within a specific tensor framework (like PyTorch), these tools remain highly efficient.

MedImages.jl offers a path forward—a future where imaging physics is integrated directly into the deep learning pipeline, where hardware acceleration is transparent and ubiquitous, and where the metadata that defines our physical world is preserved with the same rigor as the pixel values themselves. By bridging the divide between the physics of acquisition and the logic of inference, and by addressing the entwined challenges of **Volume** and **Speed** with a 7.2× faster turnaround than traditional caching frameworks [?], MedImages.jl empowers researchers and clinicians to unlock the full potential of medical imaging as a precise, quantitative science. This performance is especially critical for

Comparison with Modern Frameworks

Modern Python frameworks like MONAI [**?**] have introduced sophisticated mechanisms to couple metadata with image arrays. MONAI utilizes a `MetaTensor` structure, appending a metadata dictionary to the PyTorch Tensor. A similar design pattern appears in TorchIO, which prioritizes convenient multimodal coordination but still builds its core processing around external imaging backends. This dependency chain maintains a layer of indirection that can lead to memory management issues in large-scale ~~multi-center studies and meta-analyses, where the ability to process thousands of subjects with absolute metadata integrity enables research that was previously computationally infeasible~~training.

~~Limitations~~

~~While~~ In contrast, MedImages.jl ~~offers significant advantages in execution speed and differentiability, adopting the Juliaecosystem presents certain trade-offs. The most notable is the "time-to-first-plot" or compilation latency. Because Julia uses Just-In-Time compilation, the first execution of a complex spatial transformation incurs a compilation penalty, making initial startup slower than equivalent interpreted Python code. Furthermore, running full test suites or deploying models with heavy dependencies (e.g., CUDA.jl, Lux.jl, Zygote.jl) requires significant development memory, occasionally causing out-of-memory (OOM) issues on constrained hardware. Finally, while the Julia machine learning ecosystem is growing rapidly, it does not yet match the sheer volume of pre-trained models and plug-and-play architectural components available in the PyTorch/MONAI ecosystem.~~

5.1 ~~Limitations and Simplifications of UDE dosimetry calculation~~

~~The primary objective of this experiment was to serve as a proof of concept, demonstrating the validity and feasibility of~~ leverages Julia's type system to bind metadata directly within the `MedImage` struct. By removing the runtime overhead associated with dictionary-based metadata access, this approach contributes to the improved performance observed in our benchmarks. Furthermore, the ~~UDE-based approach in Julia~~ native implementation of core geometric transformations in Julia enables MedImages.jl to maintain the automatic differentiation graph across processing steps, which is critical for the next generation of physics-informed models in molecular imaging and quantitative theranostics. ~~While the UDE model definitively achieves the highest accuracy among the tested baselines, several physical abstractions were employed to simplify the initial implementation. To maintain scientific rigor and satisfy future clinical peer review, we acknowledge the following physical realities that the current framework abstracts for example : 1. **Static Anatomy**: The model assumes a rigid day-0 CT geometry, neglecting 4D respiratory motion. 2. **Isotropic PK**: Diffusion between neighboring voxels is currently ignored in the interstitial compartment. 3. **Receptor Dynamics**: Receptor capacity (B_{MAX}) is treated as a constant, neglecting radiation-induced receptor degradation ("stunning"). 4. **Transport Lumping**: Local β^- and penetrating γ transport are lumped into a single neural correction term rather than a dual-kernel separation.~~

~~Despite these necessary simplifications, the experiment successfully proves~~

the validity of the approach: intertwining known physics with deep learning via Julia's SciML stack yields substantially superior dosimetry results compared to pure deep learning models and classical analytical convolutions.

6 Conclusions

The development of MedImages.jl addresses core computational and metadata challenges in modern medical imaging, helping to bridge the AI chasm in nuclear medicine. By leveraging Julia's compilation architecture and type system, the library provides a unified framework where physical accuracy, computational performance, and end-to-end differentiability are achieved simultaneously. The native implementation of spatial transformations allows for transparent integration with SciML pipelines, offering a robust foundation for the next generation of physics-informed models in molecular imaging and quantitative theranostics.In practice, we envision a complementary workflow: established MONAI/SimpleITK pipelines for routine clinical deployment, and precision care and "theranostic digital twins."

Crucially, MedImages.jl resolves the historical tension between dosimetric inaccuracy and clinical intractability: while a gold-standard Monte Carlo simulation requires **48 hours** of computation to produce a single dose map [**?**], the proposed framework delivers MC-equivalent accuracy at near real-time speeds. By bridging the divide between the physics of image acquisition and the logic of AI inference, MedImages.jl for research frontiers where performance, differentiability, or algorithmic novelty are the primary requirementsempowers researchers to unlock the full potential of medical imaging as a precise, quantitative science.

Availability and Future Directions

MedImages.jl is open-source and available at `https://github.com/JuliaHealth/MedImages.jl`. All experiments described in this paper, including performance benchmarks and differentiability validations, are reproducible via the scripts provided in the repository's `experiment` directory.

Future work will focus on : Integrating integrating with advanced reconstruction frameworks (KomaMRI.jl [**?**], GIRFReco.jl [**?**]) for end-to-end physics-informed learning. Implementing and expanding support for differentiable registration algorithmsleveraging Zygote. jl. Expanding support for standardized formats like DICOM-SEG via Highdicom [**?**] integration. Developing architectural foundations for clinical-grade AI agents (e.g., VoxelPrompt) capable of multi-modal "Medical Superintelligence" benchmarks where absolute metadata integrity is a prerequisite for safe automated diagnostics. .

Declarations

Ethics approval and consent to participate: This research relies solely on retrospective analysis of fully anonymized, publicly available datasets. No new human subjects data was collected. Therefore, formal ethics committee approval and informed consent were not required. **Declaration of competing interests:** The authors declare that they have no known competing financial

interests or personal relationships that could have appeared to influence the work reported in this paper. **Funding sources:** This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. **Declaration of generative AI and AI-assisted technologies:** During the preparation of this work, the author(s) used generative AI and AI-assisted technologies for improving style and assisting in code development. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

7 Appendix

Architectural Foundations: The Coordinate System and Metadata To accurately integrate functional data with anatomical data across different spatial resolutions, software must rigorously maintain the mapping between discrete voxel indices and physical patient space. This mapping from voxel coordinates $\mathbf{v} = [i, j, k, 1]^\top$ to continuous world coordinates $\mathbf{p} = [x, y, z, 1]^\top$ relies on a 4×4 homogeneous affine transformation matrix M:

$$\mathbf{p} = M\mathbf{v}, \quad M = \begin{bmatrix} RS & T \\ \mathbf{0} & 1 \end{bmatrix} \tag{2}$$

where R is the 3×3 rotation matrix, S is a diagonal scaling matrix (voxel spacing), and T is the translation vector (origin). MedImages.jl ensures that any spatial operation $\mathcal{T}$ applied to the voxel grid is mathematically coupled with an update $M_{new} = M_{old} \times M_{\mathcal{T}}$, preventing spatial desynchronization.

Furthermore, quantitative nuclear medicine relies on precise temporal and clinical metadata. The Standardized Uptake Value (SUV), a critical metric for PET/CT, is calculated as:

$$SUV = \frac{C(t)}{ID/BW} \tag{3}$$

where $C(t)$ is the decay-corrected radioactive concentration, ID is the injected dose, and BW is patient body weight. Discarding parameters such as *Acquisition Time* or *Patient Weight* compromises the quantitative utility of the image. MedImages.jl intrinsically protects these fields within its data structures.

7.1 Workflow

When MedImages.jl offers a clear advantage for SciML. The Julia-based approach becomes compelling when research transcends standard convolutional network training and enters the realm of physics-informed models or custom numerical workflows [?]:

- **Unifying the Fractured SciML Landscape.** In Python, moving a differentiable model from PyTorch to a JAX-based differential equation solver often requires significant code rewriting or bridging overhead. Julia's multiple dispatch solves the expression problem, meaning essentially all packages are interoperable natively [?, ?]. MedImages.jl serves as the critical entry point to make this uniquely unified SciML ecosystem accessible for medical imaging.

- **Novel algorithm development.** When researchers need to implement a new spatial transformation, loss function, or physics-informed model from scratch, Julia's single-language stack eliminates the need to write C++/CUDA extensions. The algorithm can be prototyped, debugged, and GPU-accelerated within the same language, with full automatic differentiation support.

- **Cross-domain scientific computing.** Julia's ecosystem uniquely bridges medical imaging with adjacent scientific domains—DifferentialEquations.jl for pharmacokinetic modelling, KomaMRI.jl [?] for MRI pulse-sequence simulation, Flux.jl/Lux.jl [?] for deep learning—all within a single runtime. This composability enables workflows that would otherwise require fragile inter-process communication between Python, MATLAB, and C++ components.

- **Differentiable imaging physics.** Tasks such as learned registration, differentiable augmentation, or physics-informed neural networks require gradients to flow through spatial transformations. MedImages.jl's native Julia kernels integrate directly with Zygote.jl and Enzyme.jl, whereas MONAI and TorchIO must fall back on non-differentiable C++ backends for many geometric operations.

We do not advocate replacing established tools wholesale. Rather, we recommend choosing the framework that best matches the task at hand, especially when differentiability and complex scientific computing are core requirements.

Code Example

The

Code Example: Typical Workflow

The following snippet demonstrates loading a multimodal pair, rotating them simultaneously using the batched structure, and computing the SUV:

```
using MedImages

# Load PET and CT (DICOM metadata is preserved)
pet = load_image("patient_1/pet.dcm")
ct = load_image("patient_1/ct.nii")

# Create a batched tensor to process simultaneously
batch = create_batched_medimage([pet, ct])

# Rotate both volumes by 45 degrees around Z-axis (GPU supported)
rotated_batch = rotate_mi(batch, 45.0)

# Extract SUV statistics for the PET scan using an ROI mask
suv_stats = calculate_suv_statistics(rotated_batch[1], roi_mask)
println("Mean SUV: ", suv_stats.mean_suv)
```

‘‘‘

Table 4: Detailed information - Experimental Series 2: the batched multi-modal theranostic processing

Methodology	**Study-specific information**
Number of patients / datasets used (single case vs. multiple cases)	Single patient case (4 modalities)
Source and acquisition details of the imaging data (scanner type, reconstruction parameters)	[^{177}Lu]Lu-PSMA SPECT (AC/NAC), anatomical CT, and a voxel-wise absorbed dose map
Exact voxel dimensions and spatial resolution of each modality before transformation	Standardized target grids of $256 \times 256 \times 256$ and $64 \times 64 \times 64$ voxels
Whether modalities were pre-registered or inherently aligned before batching	Resampled to the common SPECT reference grid prior to batching
Definition of the rotation (angle, axis, interpolation method)	45° around the Z-axis, plus a 10-voxel translation along the X-axis
Interpolation schemes used per modality (e.g., linear, nearest-neighbor)	Linear interpolation (`Linear_en`) for continuous image data
Handling of differing voxel grids during batched transformation (internal resampling strategy)	`resample_to_image` function applied to conform differing spatial bounds
Quantitative evaluation metrics, if any, beyond visual inspection	L1 Loss, Normalized Cross-Correlation (NCC), and Mean Squared Error (MSE)
Software and hardware environment (CPU/GPU, Julia version)	Julia `Lux.jl` ecosystem with NVIDIA GPU acceleration via `CUDA.jl`
Reproducibility details (random seeds, code availability, parameter settings)	Random seed set to 42, learning rate 1e-3, available in MedImages.jl